

Edge Task Hub: Resource-Aware Edge AI Automation on Raspberry Pi

Resource-Aware Edge AI Automation on Raspberry Pi

Authors and student IDs: to be added before submission

Generated: 2026-06-01T11:11:50+00:00

Feishu is the message channel. The Raspberry Pi is the privacy and inference boundary.

Abstract

This paper presents Edge Task Hub, a Raspberry Pi based Edge AI system for scheduled reminders, daily news summaries, private document summaries, and device-health monitoring. The main research question is not "which LLM is newest"; it is "which local intelligence can run reliably on a small edge device while keeping private data local." We tested a larger 2B-size Qwen candidate, quantized Ollama models, a very small fast model, deterministic program rules, and a classical Isolation Forest model. The final design selects qwen3:1.7b for quality-first news and document summaries, keeps qwen3:0.6b for short prompts, rejects qwen3.5:0.8b for this Pi workload because of cold-start latency and empty or length-limited output, and keeps rules plus Isolation Forest because Edge AI also needs reliability and device awareness.

1. Introduction

This project was built for an Edge AI course, so the important contribution must be the edge design. A normal cloud chatbot can call a large remote model and ignore memory, storage, cold start, and privacy. Edge Task Hub does the opposite. It uses a Raspberry Pi as the decision point, runs local models when possible, records failures, and sends only finished results through Feishu.

The first version of the idea was simple: run a local model to summarize news and send reminders. That was not enough for a strong Edge AI defense. A Raspberry Pi is not a cloud GPU server. It has limited RAM, limited SD-card storage, slow cold starts, and a user who still expects the system to remind them even if a model is unavailable. The project therefore evolved from "use an LLM" into "choose the right local model or program model for each edge job."

The system now supports three user workflows. It creates scheduled reminders through Feishu text commands, generates a previous-day news summary each morning, and summarizes private files sent through Feishu. It also monitors its own CPU, memory, disk, temperature, and network metrics with a local anomaly model.

The paper focuses on experimental setup, results, and analysis, as required by the presentation instructions. Background is kept short because the course requirement is to defend the Edge AI part.

2. Edge AI Requirements

The system was designed around five edge requirements:

- Privacy: private document text should be processed on the Raspberry Pi, not uploaded to a cloud LLM.
- Memory: model selection must respect the actual RAM pressure of the device.
- Storage: a model that cannot be installed cleanly on the SD card is not a usable edge model.
- Latency policy: reminders need punctual delivery, but daily news and private document summaries can wait for better quality.
- Reliability: when an LLM fails, the device should expose the failure and still provide deterministic behavior when possible.

These requirements explain why the final project uses more than one model. Edge AI is not only about a large neural model. It is about local decision making under device constraints. In this project, LLMs handle language, program rules handle deterministic fallback, and Isolation Forest handles device health.

3. System Architecture

The final architecture separates cloud messaging from local inference. Feishu is the user interface. The Raspberry Pi is the intelligence boundary.

Local intelligence	Job	Why this belongs on the edge
qwen3:1.7b	Daily news and private document summaries	Quality matters more than strict live latency, and private text stays on the Pi.
qwen3:0.6b	Short local prompts and quick interactions	Small enough for lower-latency edge interaction when a long summary is not needed.
Rules fallback	RSS failure, missing model, or unavailable Ollama	Deterministic local behavior keeps the user workflow alive and auditable.
Isolation Forest	CPU, memory, disk, temperature, and network anomaly detection	A compact program model scores device health locally without cloud inference.

Local intelligence	Job	Why this belongs on the edge
APScheduler plus SQLite	Reminder creation and execution	The device owns the schedule and can remind through Feishu at the right time.

Current runtime service snapshot:

Service	State
edge-task-hub.service	active
ollama.service	active
openclaw-gateway.service	inactive

Current edge resource snapshot:

```
total used free shared buff/cache available
Mem: 7.6Gi 951Mi 710Mi 41Mi 6.2Gi 6.7Gi
Swap: 2.0Gi 100Mi 1.9Gi

Filesystem Size Used Avail Use% Mounted on
/dev/mmcblk0p2 29G 21G 6.3G 78% /
```

The core software stack is FastAPI, APScheduler, SQLite, Ollama, and scikit-learn. FastAPI exposes the local service and Feishu callback endpoints. APScheduler owns the reminder and summary timing. SQLite records tasks, runs, events, document summary metadata, and system metrics. Ollama runs local language models. Isolation Forest scores system-health metrics.

4. Experimental Setup

The evaluation used the deployed Raspberry Pi instead of only a laptop simulation. This matters because model behavior on the Pi is dominated by real edge limits: RAM, SD-card space, CPU speed, and cold model loading.

The tested intelligence options were:

- A larger 2B-size Qwen candidate, used to test whether a stronger LLM was practical on this edge device.
- qwen3.5:0.8b, a quantized Ollama candidate expected to fit better than the 2B-size model.
- qwen2.5:0.5b-instruct, a very small fast baseline.
- qwen3:1.7b, the final quality-first model for summary workflows.
- qwen3:0.6b, a lightweight model kept for quick prompts.
- Rules, a deterministic program model used when generative inference is unavailable.
- Isolation Forest, a classical local ML model used for device-health anomaly detection.

Application and runtime setup:

- Device: Raspberry Pi running Linux and systemd user services.
- Application stack: FastAPI, Uvicorn, APScheduler, SQLAlchemy, SQLite, and Jinja2 UI.
- AI stack: Ollama for local LLM inference and scikit-learn Isolation Forest for anomaly detection.
- Messaging stack: Feishu webhook for outbound messages and Feishu event callback for inbound commands and files.
- Candidate local models visible to the router: qwen3:1.7b, qwen3:0.6b, qwen3.5:0.8b, qwen2.5:0.5b-instruct, tinyllama, llama3.2:1b.
- Router selected backend at report generation time: ollama.
- Router selected model at report generation time: qwen3:1.7b.

Standard scheduled tasks:

Task	Type	Schedule	State
EdgeAI News Summary Demo	rss_digest	0 8 * * *	enabled
Daily Previous-Day News Summary	rss_digest	0 10 * * *	enabled

Task	Type	Schedule	State
Lunch Reminder	reminder	0 12 * * *	enabled
Sleep Reminder	reminder	30 23 * * *	enabled

5. Results

5.1 Model Exploration Logic

The model exploration followed a practical edge sequence. First, we tried to move toward a larger model because larger LLMs usually preserve facts and produce better summaries. The 2B-size candidate exposed the first edge problem: the Pi did not only need enough theoretical RAM, it also needed stable download, storage, and startup behavior. A model that is too hard to install is not a valid edge deployment choice.

Second, we tested a quantized Ollama model, qwen3.5:0.8b. Quantization is important for Edge AI because it reduces model size and memory pressure. However, the benchmark still showed about 52 to 75 seconds of cold-start generation at about 2 tokens per second, with empty or length-limited visible output. This means quantization improved deployability but did not fully solve cold-start and output-quality problems.

Third, we tested qwen2.5:0.5b-instruct. It was much faster, with a recorded 14.345-second run at 7.983 tokens per second, but summary quality was weaker and included semantic drift. This showed the opposite edge failure: a model can fit the device and still be too weak for the user-facing task.

Finally, we changed the policy. News summary is not a strict real-time generation task. The user wants a useful previous-day summary at 10:00, and the system can wait for better output. The final system therefore selects qwen3:1.7b for daily news and private document summaries, while keeping qwen3:0.6b for shorter interactions.

5.2 Model Results

Candidate	Constraint exposed	Evidence	Decision
qwen3.5:2b	Storage and deployment stability	Download attempts reached partial progress and then became unstable on the Pi, while the model blob increased storage pressure.	Rejected before final inference
qwen3:1.7b	Quality-first local summarization	Real API run on the Raspberry Pi, backend=ollama, fallback=false.	Selected
qwen3:0.6b	Low-latency lightweight prompts	Configured as OLLAMA_MODEL and listed in the resource-aware router.	Kept as fast model
qwen3.5:0.8b	Memory pressure and cold-start latency	Cold-start runs around 52.043 s to 74.832 s returned empty or length-limited visible output before the final policy.	Rejected
qwen2.5:0.5b-instruct	Quality loss from very small LLM	Fast historical benchmark, but output quality drifted and mistranslated Raspberry Pi in the news task.	Rejected for final summaries
Rules fallback	Reliability under failure	Used when Ollama is stopped, too slow, or unavailable.	Kept
Isolation Forest	Fast local metric scoring	Runs locally on CPU, memory, disk, temperature, and network metrics.	Kept

5.3 Final News Summary Result

The final verified qwen3:1.7b news-summary run used the real /api/model/news-summary endpoint on the Raspberry Pi. It returned backend=ollama, fallback=false, elapsed_seconds=55.624, and tokens_per_second=2.208.

This is not the fastest result, but it is the right result for the chosen edge workflow. Daily news summaries and private document summaries are allowed to wait. A strict short timeout would make the system look faster in a demo but would reduce summary quality and hide the real edge tradeoff.

5.4 Why qwen3.5:0.8b Cold-Started Too Slowly

The qwen3.5:0.8b issue is an important Edge AI lesson. A quantized model is smaller, but it is not automatically better for every edge workload. The recorded cold-start runs took about 52.043 seconds to 74.832 seconds, produced about 2 tokens per second, and returned empty or length-limited visible output in the news-summary style task.

The likely causes are combined:

- Cold loading: the model must be loaded into memory before useful generation starts.
- CPU generation speed: the Raspberry Pi CPU produces tokens slowly compared with a GPU or cloud service.
- Output-budget behavior: Qwen-style thinking can consume tokens before user-visible summary content appears.
- Memory pressure: one benchmark showed available memory dropping significantly during the run, which is exactly the kind of constraint an edge system must respect.

The runtime now uses `think=false` for Qwen-style models, but the final design still rejects this model for daily news summaries because the project has a better fit: `qwen3:1.7b` for waitable quality output and `qwen3:0.6b` for fast short prompts.

5.5 Feishu Automation Results

Validation item	Result
Feishu URL verification	Challenge response parser returns the exact challenge string.
Feishu text reminder	English commands create scheduled Task rows and register them with the scheduler.
Private document summary	.txt, .md, .docx, and text-based .pdf content is extracted locally before summarization.
News summary timeout policy	Slow but successful qwen3:1.7b output is accepted instead of falling back only because it exceeds 30 seconds.
Unit tests	10 project tests pass on the Raspberry Pi.
English material scan	Source comments and generated documents are checked for non-English Chinese characters before submission.

5.6 Local Device-Health Model

The Isolation Forest model is not a language model, but it is still part of the Edge AI design. It runs locally on system metrics and detects abnormal device behavior. Current anomaly evidence reports `enabled=True`, `model_trained=True`, `samples_collected=4345`, and `anomalies_24h=82`.

This is why the report describes both "large models" and "program models." Edge AI should choose the smallest reliable intelligence for the job. An LLM is useful for language summaries. Rules are useful for deterministic fallback. Isolation Forest is useful for numeric resource monitoring.

6. Discussion and Analysis

The main result is a resource-aware Edge AI architecture. A single "best model" answer would be misleading because edge devices are constrained. The best model depends on the job:

- For private documents and daily news, quality is more important than strict generation time, so `qwen3:1.7b` is selected.
- For short interactions, a smaller model such as `qwen3:0.6b` is more suitable.
- For failure cases, rules are more predictable than forcing a slow or missing LLM.
- For device health, Isolation Forest is more efficient and explainable than asking a language model to inspect numeric metrics.

The privacy argument is concrete. A user can send a private Word file or text file through Feishu. The Pi downloads it to a temporary directory, extracts text locally, summarizes it with the local model, removes the temporary file, and sends back only the summary. The sensitive document text is not sent to a cloud model.

The reminder argument is also concrete. Built-in tasks can remind the user to eat at 12:00 and sleep at 23:30, and Feishu text commands can create new reminders. The timing decision belongs to the edge device. Feishu only delivers the message.

7. Limitations and Future Work

The system still has limitations:

- The Feishu inbound app must be configured with app id, app secret, verification token, allowlists, and a public callback URL before external use.

- Local LLM inference on Raspberry Pi remains slow, especially after cold starts.
- The current PDF extraction supports text-based PDFs; scanned PDFs would need OCR, which is not part of the final verified path.
- Model quality is evaluated with project-level evidence and manual inspection, not with a large benchmark dataset.
- Student names and IDs still need to be inserted before final Canvas submission.

Future work should add a more formal quality rubric for summaries, test more quantized models under the same input, and add OCR for scanned documents if the device can handle the extra CPU cost.

8. Conclusion

Edge Task Hub demonstrates Edge AI by running useful intelligence directly on a Raspberry Pi: local LLM summaries, local private document processing, local anomaly detection, local reminder scheduling, and local fallback decisions. The model exploration matters because it shows the edge tradeoff step by step. A larger model was not deployable enough, a quantized model still cold-started poorly, a very small model was fast but lower quality, and the final system chose a hybrid model stack instead of pretending one model solves every problem.

The final defense is simple: Feishu is the communication channel, but the private processing and decisions stay on the edge device. That is the Edge AI contribution.

References

- [1] Ollama local model runtime, <https://ollama.com>
- [2] FastAPI web framework, <https://fastapi.tiangolo.com>
- [3] APScheduler documentation, <https://apscheduler.readthedocs.io>
- [4] scikit-learn IsolationForest documentation, <https://scikit-learn.org>
- [5] Raspberry Pi documentation, <https://www.raspberrypi.com/documentation/>